

The project went well. I had an easy time building the idea for the project and spent a lot of time talking with Claude through a lot of the steps before I went to build the actual project. The first place to start was just choosing a philosopher and which texts to use. I selected Aristotle because I enjoy his virtue ethics and thought many of his texts would be publicly available. I wanted to put him visually in the form of an ancient Greek style, as I thought it would suit him. Afterwards, the conversation moved to how to structure the four ideas to guide the conversation, and I wanted to connect the ideas to each other so the conversation could feel more real, rather than just have hard starts and stops. I also spent time working on the format of how he should sound, and I liked the idea of Aristotle being like a strict teacher, but still kind, given his famous role as Alexander the Great's teacher. I then went through and built the character document with Claude before writing any of the code to try and get the full picture of how I wanted Aristotle to sound before getting into the technical side. After finishing the character sheet, I spoke with Claude about how the actual code should be structured before I put the specs into a plan.md file. I had previously worked with RAG systems before and understood their general behavior, but decided that I would first try Claude's idea for how the chunking of the text should go. Then we moved on to the state machine, and I wanted to keep the logical flow of how a teacher would tutor or mentor a student. Then we came up with the idea of moving on to different topics after a student grasps the topic, and if a student fails too many times, to actually have a "bad ending" where the student has to go do more research and come back to Aristotle, as you can only pass a class if you do your homework. After this discussion, I got Claude to put everything into a plan.md file, then started building the program. I let Claude come with the first design, which was different from the examples seen on GitHub, and I appreciated the simplicity of it and got to testing. I was expecting to have to make changes to the RAG or some of the state engines, but I actually enjoyed the way Aristotle communicated and only had to fix a bug that caused it to struggle to transition between topics. After this, I got my roommate to test out the program as well to see if anything needed fixing, and they also enjoyed the way Aristotle spoke, claiming it sounded exactly like him from his readings. The only problem after they tested it out was that Aristotle was too small, and they did not notice the change in states, so I made him larger and changed his shape so he was easier to see. This project went surprisingly well, with few challenges or edits to the code, showing how impressive LLMs can be. I believe the code and final project went so smoothly because of the time I spent working on and discussing the ideas. This allows Claude to get a full picture and be descriptive when turning the conversation into the plan file. Also, turning a conversation into a plan file and spending the time getting Claude to implement phase by phase helps to make problem-solving later easier. I do not think I saw many of the downsides of LLMs in this project, but I can imagine how much more work the program would have been without taking the proper steps before writing the code.